YouTube Watch Party System

Full-Stack Architecture, Deep-Dive Security Solves & Senior Interview Guide

Candidate: Iranna Aribenchi	**Tech Stack:** React 18, TypeScript, Socket.IO, Express
Repository: github.com/Iranna0311/YouTube-Watch-Party-System	**Database:** Supabase (PostgreSQL)
Production URL: youtube-watch-party-system-udjv.onrender.com	**Test Suite:** 40 Passing Tests (Vitest)

1. High-Level Architecture & End-to-End Flow

The project enables multiple distributed users to stream YouTube videos in tight frame synchronization (<100ms drift) governed by a strict Role-Based Access Control (RBAC) hierarchy. The architecture is cleanly divided into an **ephemeral real-time state layer** (Socket.IO + in-memory Room domain models) and a **durable persistence layer** (Supabase PostgreSQL).

Layer	Technology	Responsibility
Frontend Client	React 18, Vite, TypeScript, Tailwind CSS, socket.io-client	Renders video player, NTP ping sampler, chat panel, suggestion queue, role controls.
Real-Time Server	Node.js, Express, Socket.IO, Zod, Rate Limiter	Authoritative single source of truth for video position, RBAC checks, broadcast fanout.
Persistence DB	Supabase (Managed PostgreSQL)	Durable records for created rooms, chat history, and moderation audit trails.
Hosting / CDN	Render (WebSocket Web Service) + Vercel Edge	Persistent full-duplex TCP connections on Render; global asset caching on Vercel.

2. The 4 Critical Synchronization & Security Bugs Solved

Standard YouTube sync projects suffer from subtle concurrency and authentication flaws. Here is how our codebase solves each:

Vulnerability	Flawed Standard Implementation	Our Production Solution
Bug A: Clock Skew & Double-Counting	Server advances currentTime to serverTime, and client adds (Date.now() - updatedAt) again. On every heartbeat, client advances ahead of the host.	**NTP Clock Sync:** Client measures roundtrip latency and clock offset: $\Delta = t_{server} - (t0+t1)/2$. applySyncState strictly computes authoritative time relative to serverTime + clockOffset.

Bug B: Reentrancy Lock Deadlock	isRemoteUpdate flag is only cleared inside onStateChange. If player is already in target state, no event fires, permanently locking user controls.	**Self-Clearing Cooldown:** applySyncState arms a 500ms fallback timeout that forcibly clears isRemoteUpdateRef, ensuring user controls never freeze.
Bug C: Host Invariant Fracture	Host promotes participant to 'Host' via assignRole. This changes role in map but leaves hostId pointing to old user, fracturing room ownership.	**Invariant Enforcement:** Room.assignRole explicitly rejects 'Host'. All ownership changes must pass through Room.transferHost() atomically.
Bug D: Identity Hijacking	join_room reconnects using public userId from participant list. Any malicious participant can replay Host's userId and steal admin privileges.	**Cryptographic Session Tokens:** Private UUID token generated on connect, stored in memory, and stripped from all public broadcasts. Reclaim requires matching sessionToken.

3. Video Queue Auto-Advance & Anti-Abuse Debouncing

The Problem: When a YouTube video finishes, 10+ clients connected to the same room will emit video_ended at nearly the exact same millisecond. Without deduplication, the server would skip 10 songs ahead in the queue. Furthermore, malicious Participants could forge video_ended packets to skip songs they dislike.

The Solution in Room.ts:

• **Role Guard:** Only Host or Moderator can trigger queue advances (this.can(userId, 'manage_queue')).
• **State Match:** The reported videoId must strictly match this.videoState.videoId.
• **3000ms Time Window Debounce:** if (Date.now() - this.lastAdvanceAt < 3000) return null;. The first valid signal advances the queue and stamps lastAdvanceAt = Date.now(). All subsequent simultaneous client signals are dropped.

4. Concurrency Load-Test Benchmarks & Fanout Mathematics

The server was benchmarked using an automated WebSocket load-test suite (npm run test:load) simulating **25 rooms × 10 users = 250 concurrent WebSocket connections** emitting high-frequency playback barrages:

Metric	Measured Result	Engineering Explanation
Connections Attempted / Succeeded	250 / 250 (100%)	Zero dropped handshakes or connection timeouts under load.
Failures / Dropped Packets	0 (0.00%)	Sliding-window token bucket (15 req/s) absorbed valid traffic without dropping.
Events Emitted / Received	325 / 1,975	Exact deterministic broadcast arithmetic (detailed below).
Round-Trip Event Latency	Avg: 24.89ms | p95: 44ms	Well within the 100ms threshold for human imperceptible video sync.
Server Process Heap Delta	+7.30 MB (Total: 18.38 MB)	Queried live via /health endpoint; isolates server memory from test runner.

The Exact Mathematical Proof for 1,975 Events:

1. **Initial Room Joins:** 25 rooms × 9 joiners = **225** initial sync snapshots.
2. **Playback Action Fanout:** 25 hosts each emit 3 actions (play, seek, pause = 75 actions). For each action, the server broadcasts both the specific event AND the authoritative sync_state snapshot to all 10 room members: 25 × 3 × 2 × 10 = **1,500** receipts.
3. **NTP Latency Pings:** 250 connected sockets each receive 1 ping response = **250** receipts.

Total Fanout: 225 + 1,500 + 250 = **1,975 receipts**.

5. Top 15 Technical Interview Questions & Senior Answers

Prepare these answers for technical interviews and architectural code walkthroughs:

Q1: Why did you choose WebSockets over HTTP polling or Server-Sent Events (SSE)?

A: Video watch parties require full-duplex bidirectional streaming with sub-100ms latency. HTTP polling incurs heavy HTTP header overhead (1KB per poll) and introduces latency intervals. SSE is unidirectional (server-to-client only), requiring separate HTTP POST requests for client actions like seek or pause. WebSockets establish a single persistent TCP connection with a lightweight 2-to-6 byte framing overhead, enabling instantaneous bidirectional broadcast.

Q2: How does the server guarantee it remains the single source of truth for video time?

A: The server maintains an internal VideoState { playState, currentTime, updatedAt }. When playing, the server does not tick an internal timer on every frame; instead, it dynamically computes authoritative time on demand: currentTime + (Date.now() - updatedAt)/1000. Clients never tell the server what time it should be; clients send requests (e.g., 'seek to 45s'), the server validates RBAC permissions, updates its state timestamp, and broadcasts the authoritative snapshot.

Q3: How do you prevent event echo loops when broadcasting playback state?

A: When user A scrubs the player, their client emits a 'seek' event. The server broadcasts this seek to all room participants, including user A. If user A's player blindly reacts to incoming server events, it would fire another seek event, triggering an infinite broadcast loop. We solve this by setting an isRemoteUpdateRef flag before calling player.seekTo(). Additionally, we attach a 500ms self-clearing timer fallback so user controls are never permanently deadlocked if YouTube fails to fire a state change.

Q4: Why does the YouTube IFrame API not provide a native 'seek' event, and how did you detect scrubbing?

A: YouTube's IFrame API only fires onStateChange (PLAYING, PAUSED, BUFFERING, ENDED) without a dedicated onSeek event. To capture scrubbing, our custom hook runs a 500ms polling interval comparing player.getCurrentTime() against the previously recorded time. If the delta exceeds 1.5 seconds while playState remains unchanged, we know the user scrubbed the progress bar and emit a 'seek' event.

Q5: How do you prevent clock drift between clients on different operating systems?

A: Clients may have system clocks skewed by seconds or minutes. Comparing Date.now() directly against the server's updatedAt causes drift. We implement NTP-style clock synchronization: on connect and every 30 seconds, the client emits time_ping with timestamp t0. The server echoes serverTime. Upon response at t1, the client calculates clockOffset = serverTime - (t0 + t1)/2. All elapsed time calculations strictly reference (serverNow = Date.now() + clockOffset).

Q6: How does your Role-Based Access Control (RBAC) work, and how is it enforced?

A: We use an Object-Oriented Permissions class with a static Map defining allowed actions per role: Host (all permissions, transfer host, kick, assign roles), Moderator (play/pause, seek, change video, manage queue), and Participant (view only). Every socket event passes through room.can(userId, action) before executing. Unauthorized events return an immediate error event. UI buttons are conditionally disabled for UX, but the backend never trusts client-side validation.

Q7: Explain the Bug D vulnerability (Identity Hijacking) and your cryptographic solution.

A: In naive WebSocket watch parties, reconnecting users identify themselves using userId stored in localStorage. However, the participant list broadcasts all userIds publicly. A malicious participant could copy the Host's userId from dev tools, emit join_room with that userId, and take over host privileges. We solved this by issuing a private randomUUID() sessionToken upon room creation. This token is returned exclusively to the socket, never broadcasted, and stripped from toJSON() payloads. Reclaiming a session requires matching both userId and sessionToken.

Q8: What happens when the Host abruptly disconnects or refreshes the page?

A: If the Host loses connection, we don't immediately disband the room or demote them. The server triggers a 10-second grace period timer. If the Host reconnects within 10 seconds presenting their valid sessionToken, they reclaim their Host socket seamlessly. If the timer expires without a reclaim, the server auto-promotes the oldest connected Moderator (or oldest Participant if no moderators exist) and broadcasts the role change and audit log.

Q9: How do you handle kicked or banned users to prevent them from rejoining?

A: When the Host kicks a participant, the server adds their userId to a per-room bannedUserIds Set and forcibly calls socket.disconnect(true). Any subsequent join_room event matching that userId is immediately rejected with an error. In Known Limitations, we acknowledge that in-memory bans can be bypassed if an anonymous user clears localStorage/cookies, and in production, this should be anchored to authenticated user accounts.

Q10: Why do you store room state in-memory rather than querying Supabase on every WebSocket tick?

A: High-frequency video sync events occur hundreds of times per minute across multiple rooms. Writing play/pause/seek ticks to a relational database like PostgreSQL on every second would exhaust database connection pools, introduce 20-50ms disk I/O latency, and create database bottlenecks. The industry standard is keeping real-time sync in-memory/WebSockets and snapshotting durable milestones (room creation, video change, chat, audit logs) to Supabase asynchronously.

Q11: What are the architectural trade-offs of using Socket.IO Redis Adapter for horizontal scaling?

A: @socket.io/redis-adapter uses Redis Pub/Sub to relay broadcasts between multiple Node instances. However, our Room objects and participant Maps live in each server instance's RAM. If User A connects to Instance 1 and creates room 'ABC123', User B connecting to Instance 2 will receive 'Room not found' unless sticky routing (consistent IP or room code hashing at the load balancer) routes all participants of room 'ABC123' to the same instance, or room domain models are externalized to Redis Hashes.

Q12: How do you protect the server from WebSocket flood attacks or DDoS?

A: We implement a two-layer defense: First, every socket connection is bounded by a SocketRateLimiter implementing a sliding-window token bucket capped at 15 requests/second per socket ID. Second, every incoming payload is validated using Zod schemas (validating 11-char YouTube ID regex, finite non-negative seek timestamps, trimmed string lengths). Floods are dropped before touching the Room domain model.

Q13: How did you solve the browser autoplay policy restriction?

A: Modern browsers (Chrome, Safari) block video playback with audio unless preceded by user interaction (gesture/click). If a late-joining participant connects to a room where the video is already playing, calling player.playVideo() directly triggers an unhandled DOMException error. We handle this in React with an overlay 'Click to Join Watch Party' banner, ensuring a valid user gesture enables the YouTube audio context.

Q14: Explain your suggestion queue and approval workflow.

A: Instead of only letting Hosts change videos, Participants can submit video URLs as a change_request. The server stores requests in pendingRequests and broadcasts change_request_received exclusively to staff (Host and Moderators). When approved, the video is added to videoQueue. When the active video ends, tryAdvanceOnVideoEnd automatically plays the next queued video with a 3-second debounce window.

Q15: How does your monolithic Render deployment avoid CORS issues entirely?

A: In production, the compiled server (server/dist/index.js) checks if client/dist exists and serves the built Vite bundle statically via express.static(), paired with an SPA catch-all (app.get('*')). The frontend client connects to window.location.origin (io('/')). Because the frontend and WebSocket backend share the exact same host and port, cross-origin resource sharing (CORS) is eliminated entirely.